

PRACTICAL-3

Aim : To Implementation of Max-Heap sort algorithm

Algorithm :

Heap Sort Algorithm :-

Input : Array A with n elements
Output : Sorted array A with in ascending order

- 1) start
- 2) read n and array A
- 3) set heap-size : n
- 4) set $i = \text{floor}(n/2)$
- 5) Repeat while $i \geq 1$
 - * set Parent = i
 - * Repeat while $(2 * \text{Parent}) \leq \text{heap-size}$
 - * set left = $2 * \text{Parent}$
 - * set right = left + 1
 - * set largest = Parent
 - * If $A[\text{left}] > A[\text{largest}]$: set largest = left
 - * If right $\leq \text{heap-size}$ and $A[\text{right}] > A[\text{largest}]$: set largest = right
 - * If largest $\neq \text{Parent}$, Break loop
 - * swap $A[\text{Parent}]$ and $A[\text{largest}]$
 - * set Parent = largest
 - * Decrement i by 1
- 6) set $i = n$
- 7) Repeat while $i \geq 2$
 - * swap $A[1]$ and $A[i]$
 - * set heap-size = heap-size - 1
 - * set Parent = $i/2$
 - * Repeat while $(2 * \text{Parent}) \leq \text{heap-size}$
 - * set left = $2 * \text{Parent}$

```

* Set right = left + 1
* Set largest = Parent
* If A[left] > A[largest], set largest = left
If right != heap-size and A[right] > A[largest]
    set largest = right
* If largest == Parent break loop
* swap A[Parent] and A[largest]
* Set Parent = largest
* Decrement i by 1
8) Display sorted array A
9) Stop
    
```

Program / Code :

```

#include <iostream>
using namespace std;
    
```

```

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
    
```

```
heapify(arr, n, i);
for (int i = n - 1; i > 0; i--)
{
    int temp = arr[0];
    arr[0] = arr[i];
    arr[i] = temp;

    heapify(arr, i, 0);
}
}

int main()
{
    int n;
    int arr[100];

    cout << "Enter the number of elements: ";
    cin >> n;

    cout << "Enter " << n << " elements:\n";

    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }

    cout << "\nOriginal array: ";

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

    heapSort(arr, n);

    cout << "\nSorted array: ";

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

Output :

```
Enter the number of elements: 5
Enter 5 elements:
3
9
23
15
7

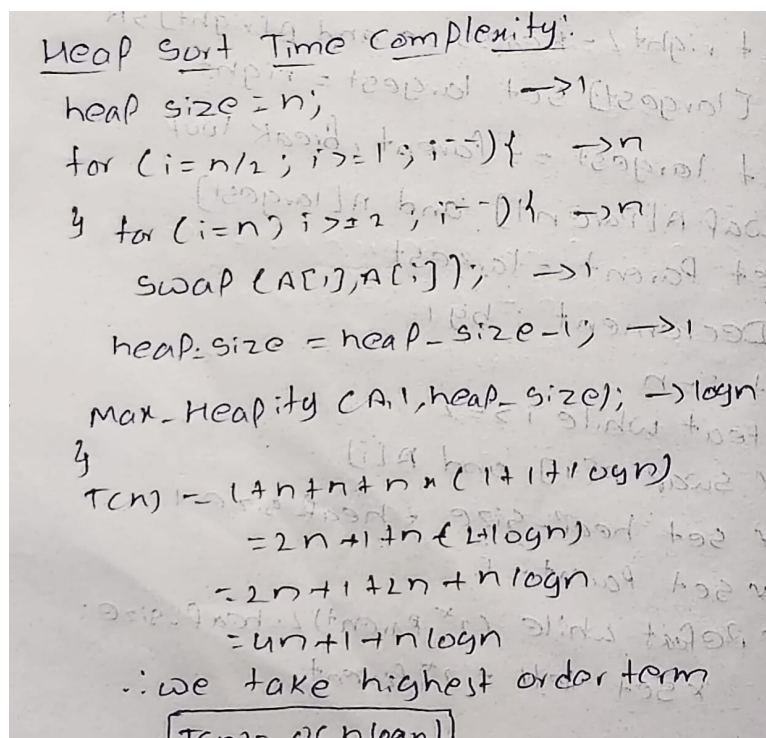
Original array: 3 9 23 15 7
Sorted array: 3 7 9 15 23
```

Time Complexity :

Best Case : $O(n \log n)$

Average Case : $O(n \log n)$

Worst Case : $O(n \log n)$



Heap sort Time complexity:

heap size = n ;
for ($i = n/2$; $i >= 1$; $i--$) {
 for ($j = n$; $j >= 2*i$; $j--$) {
 swap ($A[i], A[j]$);
 }
 max-Heapify (A, i , heap-size);
 heap-size = heap-size - 1;
}

$T(n) = 1 + n + n * (1 + 1 + \log n)$
 $= 2n + 1 + n(1 + \log n)$
 $= 2n + 1 + 2n + n \log n$
 $= 4n + 1 + n \log n$
 $\therefore$ we take highest order term
 $T(n) = O(n \log n)$

1.BEST CASE:

Best case: $T(n) = O(n \log n)$
→ This occurs if the array is already sorted or configured in a way that minimizes swaps
⇒ $T(n)$ Time complexity becomes $O(n)$, when all elements are identical
eg: [7, 7, 7, 7]
→ The algorithm looks at the parent node & compares them to their children
→ If parent is smallest than child it is not
As a result 'zero' swaps are made

2.AVERAGE CASE:

Average case: $T(n) = O(n \log n)$
→ This occurs when the elements are in completely random, jumbled order

3.WORST CASE:

Worst case: $T(n) = O(n \log n)$
→ This occurs when the distinct elements are arranged such that every single time you swap a leaf element to the root it happens to be smaller than all its children

Conclusion : The Max-Heap Sort algorithm was successfully implemented and executed to sort the elements in ascending order. The program verified a consistent time complexity of $(O(n \log n))$ across the best, average, and worst cases using an in-place binary heap structure.